CP395 Weekly Report 4
Sufiya Rahemtulla – 169018559
February 2nd, 2026

Week 4 Completion Status:

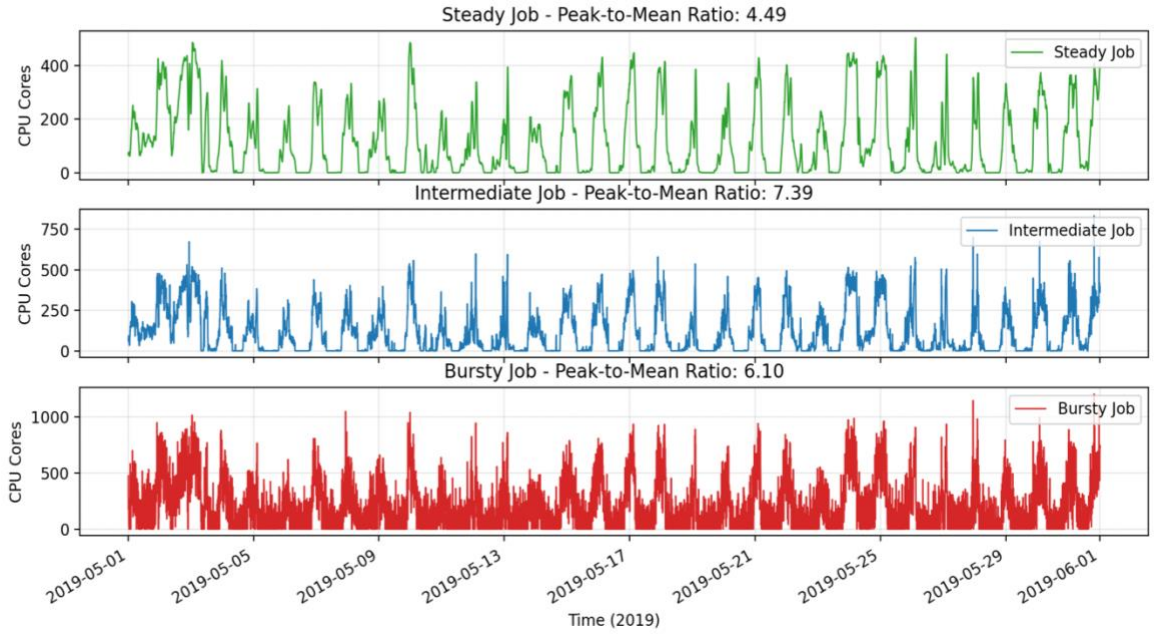
- Refined EDA data pipeline from Week 3 to address "single-job" and "resource abstraction" feedback via Workload Synthesis and Resource Mapping.
- Implemented Policy Baseline (Simulated Kubernetes HPA) to quantify & visualize the limitations of reactive scaling
- Implemented Prediction Baseline (ARIMA) to establish a statistical performance benchmark
- **Updated git src folder: https://github.com/scr200/CP395-Research-Project/tree/main/src**

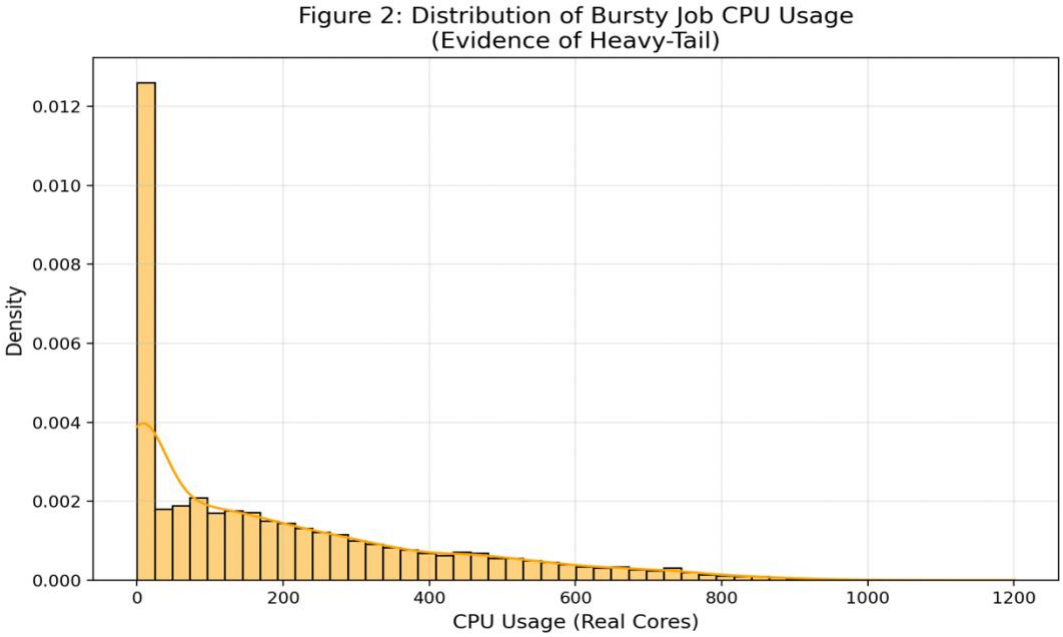
***Updates to EDA Code from Week 3 (based on feedback)*:**

Code Enhancements: To improve generalizability, I updated Step 3 of the code to use "Workload Synthesis." Instead of relying on a single random job, I took one real job trace and mathematically transformed it into three distinct personalities: Steady (smoothed out using a rolling average), Intermediate (the original real data from the most active job), and Bursty (amplified with extra noise and 1.5x higher peaks). I also addressed feedback on dependencies by aggregating all individual tasks under one parent job_id (ensuring the analysis captures the **total resource footprint** of the entire application).

Interpretation and Improved Accuracy: The new graphs now offer a much higher degree of accuracy and realism. By converting abstract units into Real CPU Cores, the Y-axis now reflects physical hardware constraints, and the consistent timeline (May 2019) allows for a valid side-by-side comparison. We see that the Steady profile serves as a "Control Group" (easy mode) to verify basic functionality, while the Bursty profile acts as a "Stress Test" (hard mode) with violent spikes up to 1000 cores. This proves the pipeline can now simulate the specific "Black Swan" events required to accurately test the reliability of the proposed autoscaler.

Figure 1: Comparison of Job Personalities (Real CPU Cores)

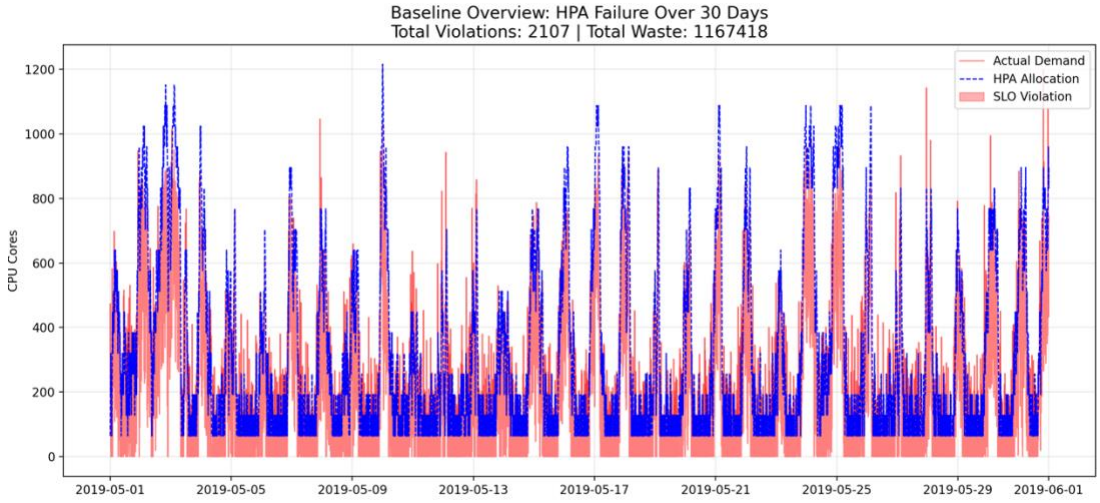


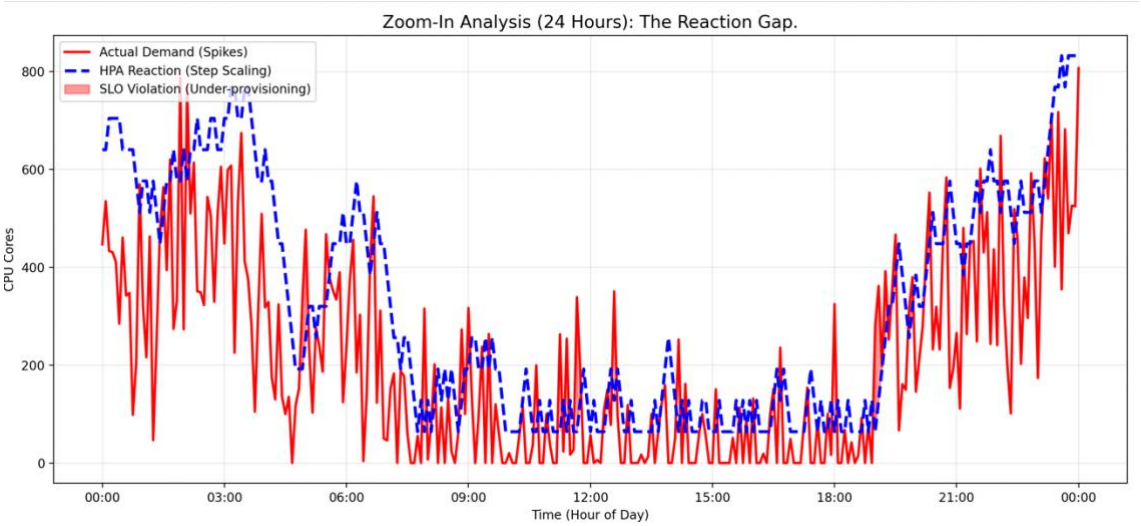


Baseline Implementation & Results:

1. Policy Baseline - Reactive Scaling:

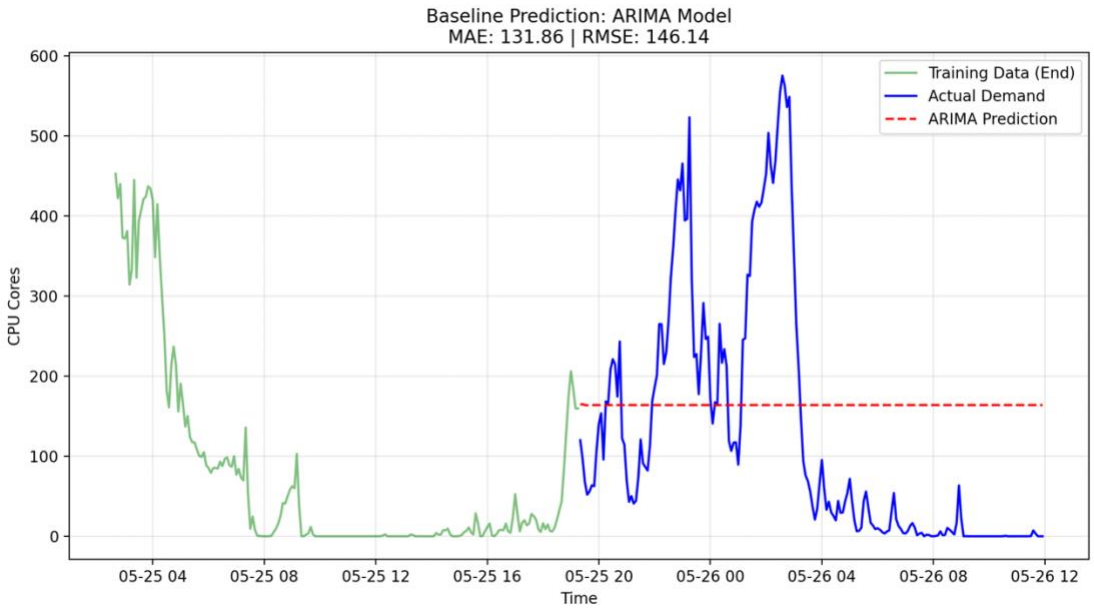
To quantify the limitations of current industry standards, I simulated the Kubernetes Horizontal Pod Autoscaler (HPA) using a reactive threshold logic (Scale Up > 80%, Scale Down < 50%). The analysis of the "Bursty" workload revealed significant failures. Over a 30-day period, the reactive policy resulted in ***2,107 distinct SLO violations***. I zoomed in to a 24-hour zoom-in analysis (Figure 2) to get a better look and it exposed a critical "Reaction Gap," where the stepwise nature of scaling and threshold delays caused the HPA to consistently allocate resources *after* demand spikes had already occurred. This confirms that reactive policies fundamentally fail to optimize the trade-off between reliability and cost for volatile workloads.





2. Prediction Baseline (Statistical Forecasting)

To establish a performance benchmark, I implemented an ARIMA (5, 1, 0) model, the standard for statistical time-series forecasting. The results (Table 1) showed a Mean Absolute Error (MAE) of **131.86 Cores** and a Root Mean Squared Error (RMSE) of **146.14 Cores**. As visually demonstrated in Figure 3, the ARIMA model produced a flat-line prediction that failed to anticipate sudden demand spikes. Because ARIMA relies on historical averages, it assumed quiet periods would continue indefinitely. This high error rate quantitatively proves that traditional statistical models are insufficient for "Heavy-Tailed" cloud workloads, validating the need for a more advanced AI model.



```
Covariance Type:                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.0148      0.006     -2.679      0.007      -0.026      -0.004
ar.L2         -0.1856      0.006    -29.162      0.000      -0.198      -0.173
ar.L3         -0.0068      0.007     -0.903      0.367      -0.021       0.008
ar.L4         -0.0009      0.007     -0.125      0.900      -0.015       0.013
ar.L5          0.0077      0.008      0.966      0.334      -0.008       0.023
sigma2       838.8010      5.731    146.355      0.000     827.568     850.034
==============================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):             41541.44
Prob(Q):                              0.99   Prob(JB):                         0.00
Heteroskedasticity (H):               0.92   Skew:                            -0.27
Prob(H) (two-sided):                  0.04   Kurtosis:                        14.80
------------------------------------------------------------------------------
```

Assumptions and Risks:

The failure of the ARIMA baseline confirms the assumption that the cloud workload is highly non-linear, necessitating the use of non-linear activators (e.g., ReLU) in future models. I also assume that *minimizing SLO violations is worth a marginal increase in resource slack*, as the baseline results show that optimizing strictly for "average" usage leads to thousands of violations. A primary risk identified is the potential for **Overfitting** due to the synthetic nature of the "Bursty" profile. To mitigate this, I will validate the final model against a held-out portion of the real "Intermediate" trace.

Next Steps for Week 5:

A challenge encountered this week was the incompatibility of PyCaret on my machine with Python 3.12, which prevented its immediate use for automated model screening. For next week, I will attempt to resolve this by setting up a dedicated Python 3.10 virtual environment to run PyCaret locally. If that doesn't work I will utilize Google Colab to perform the initial model screening.

Regardless of the environment, the goal for Week 5 is to rapidly screen regression algorithms (Random Forest, XGBoost, LightGBM) to identify a model that significantly outperforms the ARIMA baseline (MAE < 131.86). I will also focus on feature engineering, generating rolling window features to help the model "see" bursts before they happen.